Block Categorical Flow Maps for Few-Step Semi-Autoregressive Text Generation

Sergei Kholkin[1]
[1] Skolkovo Institute of Science and Technology

Abstract

Autoregressive language models generate an L-token sequence in L sequential steps, whereas categorical flow maps (CFMs) update all positions jointly in one or a few network evaluations but are tied to a fixed full-sequence context. We investigate Block Categorical Flow Maps (BCFM), which apply a conditional CFM block by block while reusing a finalized clean prefix. On GPT-2-tokenized TinyStories with $L = 256$, we compare a full-sequence CFM (M1), a training-free blockwise wrapper (M2), and a block-conditional fine-tune with block size $B = 16$ (M3), together with MDLM and BD3-LM baselines. Quality is measured on 512 generations with MAUVE and GPT-J-6B generative perplexity; all latency is measured on one H100 NVL. The maximum-speed M3 sampler reaches its best MAUVE, 0.7550, at NFE 16, with generative perplexity 58.13 and 190.1 ms batch-one latency. A 200k-step M1 obtains MAUVE 0.6577 and perplexity 67.49 at NFE 32 in 133.8 ms, while M2 never exceeds MAUVE 0.0267. Thus learned block conditioning is essential and improves the CFM-only quality frontier. However, MDLM NFE 16 reaches MAUVE 0.9157 and perplexity 26.99 in only 128.0 ms, strictly dominating the best-MAUVE M3 point under the matched protocol.

Index Terms. Discrete generative models, flow maps, semi-autoregressive generation, few-step sampling.

1 Introduction

Autoregressive generation is the dominant paradigm for text [5, 9]. Tokens are produced left-to-right, so generating L tokens costs exactly L sequential network evaluations. This sequential decoding is the core latency bottleneck.

Discrete diffusion models [7, 11, 12] instead corrupt the whole sequence with noise and learn to denoise it in parallel, reducing the cost to $N \leq L$ refinement steps. But N can still be large, so parallel decoding alone is not fast enough.

Categorical flow maps [6, 8, 10] distill such a denoiser into a one- (or few-) step noise-to-data generator on the probability simplex, collapsing the multi-step trajectory into a single jump and bringing N down to 1–4.

However, standard CFM generates one full sequence at once, at a fixed length. Semi-autoregressive (block-by-block) generation offers a complementary structure: it can expose a cacheable clean prefix, support variable-length outputs, and reduce the number of positions generated jointly. This structure is used by recent discrete-diffusion systems [1, 2, 4].

Problem. The standard CFM formulation gives few-step sampling only as a monolithic full-sequence jump. It commits to all L tokens from a single prior draw, has no explicit causal prefix, and therefore cannot cache finalized context or extend a sequence beyond its training length. Block-autoregressive diffusion (BD3-LM) provides a causal, cacheable, length-flexible structure, but each block is still produced by a many-step masked-diffusion reverse process. Directly applying flow-map consistency objectives is non-trivial because they are formulated for continuous deterministic trajectories rather than discrete stochastic reverse transitions.

Approach and contributions. We realize categorical flow maps as semi-autoregressive models, *Block Categorical Flow Maps* (BCFM), by keeping BD3-LM's block factorization and clean-prefix conditioning while replacing each block's masked diffusion with a *conditional categorical flow map* from a simplex prior to the block tokens. We study two adaptations. The **inference-time** variant runs an existing full-sequence CFM block by block without further training. The **training-time** variant fine-tunes the network as a block-conditional flow map with blockwise flow-matching and self-distillation losses (Sec. 3). Our TinyStories experiments show that the training-free wrapper fails, whereas learned block conditioning substantially improves both MAUVE and external-model perplexity. We also report the complete quality–latency curves, identify a non-monotonic refinement optimum at NFE 16, and compare BCFM with matched-scale MDLM and BD3-LM baselines.

2 Background

From autoregression to discrete diffusion. An autoregressive model produces text one token at a time, so generating L tokens always costs L sequential forward passes. Discrete diffusion [7, 11] breaks this serial constraint by enabling *multi-token* generation. A forward process progressively corrupts the whole sequence (replacing tokens with a `[MASK]` symbol, or driving the categorical state toward a known prior), and a learned denoiser inverts it, revising many positions at once. Generation then takes $N \leq L$ parallel refinement steps rather than L sequential ones. Quality typically improves with N, at the cost of additional evaluations in methods such as MDLM, SEDD, and the Diffusion Duality (Duo) [12].

Categorical and variational flow matching. A second route to parallel token decoding is to leverage flow-based generative models, which cast denoising as a *flow* between noise and data. Realizing such a flow over discrete tokens requires a continuous notion of partial corruption, which

Variational Flow Matching (VFM / CatFlow) [3] supplies. Let $x = (x_1, \dots, x_L)$ be a sequence with $x_i \in \{1, \dots, K\}$, let $\delta_x \in (\Delta^{K-1})^L$ be its one-hot embedding, and let $z_0 \sim p_0$ be a continuous prior on the product of simplices. VFM connects noise to data by the straight-line interpolant

$$z_t = \underbrace{(1-t)\, z_0}_{\text{prior}} + \underbrace{t\, \delta_x}_{\text{data}}, \qquad t \in [0,1], \tag{1}$$

which runs from pure noise z_0 at t=0 to the clean data $z_1 = \delta_x$ at t=1 and stays in $(\Delta^{K-1})^L$ throughout. Rather than regressing a velocity directly (awkward on the simplex), it learns the *mean-field posterior* over the clean tokens given the noisy state,

$$q_\theta(x \mid z_t) = \textstyle\prod_{i=1}^{L} q_\theta(x_i \mid z_t), \tag{2}$$

in which each factor $q_\theta(x_i \mid z_t)$ is a categorical over $\{1, \dots, K\}$ trained by per-token cross-entropy. Stacking these marginals gives the *denoiser* $q_{t,t}(z_t) \in (\Delta^{K-1})^L$, with entries $[q_{t,t}(z_t)]_{i,k} = q_\theta(x_i{=}k \mid z_t)$, from which the marginal generative velocity follows in closed form,

$$b_t(z) = \frac{q_{t,t}(z) - z}{1-t}. \tag{3}$$

Sampling then draws $z_0 \sim p_0$ and integrates the ODE $\dot{z}_t = b_t(z_t)$ from t=0 to 1 with N Euler steps on a grid $0 = \tau_0 < \cdots < \tau_N = 1$,

$$z_{\tau_{m+1}} = z_{\tau_m} + (\tau_{m+1} - \tau_m)\, b_{\tau_m}(z_{\tau_m}), \tag{4}$$

each step costing one evaluation of the denoiser $q_{t,t}$; the tokens are read off from the endpoint as $x_i = \arg\max_k [z_{\tau_N}]_{i,k}$.

Categorical flow maps. The ODE (4) still has to be integrated over many steps. Categorical Flow Maps (CFM) [10], and the related discrete flow maps and one-step continuous-denoising models [6, 8], remove that cost by distilling the field into a *flow map* $X_{s,t}$ that jumps directly from time s to a later time t. CFM generalizes the diagonal denoiser $q_{t,t}$ to a two-time head $q_{s,t}(z_s) \in (\Delta^{K-1})^L$ (the clean-token distribution predicted from state z_s when targeting time t, which reduces to the VFM denoiser on the diagonal $s = t$), and parametrizes the map through this endpoint ("partial-denoiser") prediction,

$$\begin{aligned} X^\theta_{s,t}(z_s) &= (1-\gamma_{s,t})\, z_s + \gamma_{s,t}\, q^\theta_{s,t}(z_s), \\ \gamma_{s,t} &:= \frac{t-s}{1-s}. \end{aligned} \tag{5}$$

The map is pinned down by two conditions, a *tangent condition* $\partial_t X_{s,t}\big|_{t=s} = b_s(z_s)$ recovering the VFM velocity (3), and a *Lagrangian condition* $(1-t)\,\partial_t X_{s,t} = q_{t,t}(X_{s,t}) - X_{s,t}$. CFM enforces them by self-distillation, adding to the VFM endpoint loss an Endpoint-Consistent Lagrangian Distillation (ECLD) term over $s < t$ that forces the few-step map to agree with the model's own instantaneous denoiser (requiring a forward-mode / JVP derivative of $q_{s,t}$ in t). Sampling then replaces the N Euler steps of (4) with a few jumps of the map. We draw $z_0 \sim p_0$ and, over a short grid $0 = s_0 < s_1 < \cdots < s_n < s_{n+1} = 1$ with n small, iterate

$$z_{s_{j+1}} = X_{s_j,\, s_{j+1}}(z_{s_j}), \qquad j = 0, \dots, n, \tag{6}$$

followed by $x_i = \arg\max_k [z_1]_{i,k}$; the one-step case $n = 0$ is the single jump $z_1 = X_{0,1}(z_0)$. The reference implementation is available at:

`https://github.com/olsdavis/semicat`

Block-autoregressive diffusion. Everything so far treats the sequence as one monolithic, fixed-length object. Block diffusion (BD3-LM) [1] reintroduces autoregressive structure at the level of *blocks*. For $M = L/B$ blocks of size B, it uses the exact factorization

$$p_\theta(x) = \prod_{b=1}^{M} p_\theta\left(x^{(b)} \mid x^{(<b)}\right). \tag{7}$$

Each conditional is a masked diffusion over block b given the *clean* previous blocks $x^{(<b)}$. A single block-causal attention mask over $[x_{\text{clean}}; x_{\text{noisy}}]$ yields all block losses in one forward pass; at inference the finalized blocks form a KV cache and the generation length is unconstrained, though each block still pays the full multi-step diffusion cost. This recipe underlies recent systems including LLaDA 2.0 [2] and DiffusionGemma [4]. BCFM keeps the factorization in Eq. (7) while replacing the per-block diffusion with CFM's few-step jump.

3 Methods

Block condition and probabilistic model. For block b the condition is the finalized, *clean* discrete prefix

$$c^{(b)} := x^{(<b)} = \left(x^{(1)}, \dots, x^{(b-1)}\right). \tag{8}$$

BCFM realizes each conditional factor by a conditional categorical flow map:

$$\begin{aligned} z_0^{(b)} &\sim p_0^{\otimes B}, \\ z_1^{(b)} &= X^\theta_{0,1}\left(z_0^{(b)}; c^{(b)}\right), \\ x^{(b)} &= \mathcal{D}\left(z_1^{(b)}\right). \end{aligned} \tag{9}$$

where $\mathcal{D}$ discretizes the endpoint by taking an argmax or sampling from $q^\theta_{\cdot,1}$, and the conditional map is Eq. (5) with the head $q^\theta_{s,t}(z_s^{(b)}, c^{(b)})$ now reading the prefix. Each block carries its own time pair (s_b, t_b), injected per token via adaLN modulation with two time embeddings. The limiting cases provide consistency checks: $B = 1$ is an AR latent-noise model, whereas $B = L$ recovers full-sequence CFM. Intermediate values of B trade sequential block depth against the difficulty of joint within-block generation.

Training objective. Training conditions block b on the gold prefix $c^{(b)}$. The per-block sampling law is most compactly written as

$$\begin{aligned} x &\sim p_{\text{data}}, \qquad & t_b &\sim \mathcal{U}(0, 1-\varepsilon), \\ s_b &\sim \mathcal{U}(0, t_b), \quad & z_0^{(b)} &\sim p_0^{\otimes B}, \end{aligned} \tag{10}$$

and $z_u^{(b)} = (1-u) z_0^{(b)} + u\delta_{x^{(b)}}$ for $u \in \{s_b, t_b\}$. We write expectation under Eq. (10) as $\mathbb{E}_b$. The per-block *flow-matching*

(VFM endpoint) loss is the mean-field cross-entropy of recovering the clean block from its noised copy at the diagonal time $s = t$.

$$\mathcal{L}^{(b)}_{\mathrm{VFM}}(\theta) = -\frac{1}{B}\,\mathbb{E}_b\left[\sum_{i=1}^{B} \log q^{\theta}_{t_b,t_b}\left(x_i^{(b)} \mid z_{t_b}^{(b)}, c^{(b)}\right)\right]. \quad (11)$$

The per-block *self-distillation* (ECLD) loss enforces the flow-map conditions over $s_b < t_b$,

$$\bar{q}_b := \mathrm{sg}\left[q^{\theta}_{t_b,t_b}\left(X^{\theta}_{s_b,t_b}(z_{s_b}^{(b)}; c^{(b)})\right)\right], \quad (12)$$

$$\mathcal{L}^{(b)}_{\mathrm{ECLD}}(\theta) = \mathbb{E}_b\left[\frac{4}{(1-t_b)^2}\,\mathrm{CE}\Big(\bar{q}_b, q^{\theta}_{s_b,t_b}(z_{s_b}^{(b)}; c^{(b)})\Big)\right] + 2\,\mathbb{E}_b\left[\gamma^2_{s_b,t_b}\left\|\partial_t q^{\theta}_{s_b,t_b}(z_{s_b}^{(b)}; c^{(b)})\right\|_2^2\right]. \quad (13)$$

with sg denoting stop-gradient. The total objective averages both terms over blocks,

$$\mathcal{L}(\theta) = \frac{B}{L}\sum_{b=1}^{L/B}\left[\mathcal{L}^{(b)}_{\mathrm{VFM}}(\theta) + \lambda\,\mathcal{L}^{(b)}_{\mathrm{ECLD}}(\theta)\right], \quad (14)$$

and all L/B block losses are obtained in *one* masked forward pass. Because each block's targets read only its own noisy state and the clean prefix $c^{(b)}$, the per-block stop-gradient targets stay mutually independent, so the next block's loss is well-defined given the previous blocks' clean condition.

Target training-time adaptation. The formulation in Eq. (14) is the intended ECLD-based BCFM objective. In this construction, the clean and noisy streams are concatenated into a doubled sequence $[\,x_{\mathrm{clean}}; z_{\mathrm{noisy}}\,]$ of length $2L$, over which attention runs with a single $2L \times 2L$ block-causal mask. All L/B block losses can therefore be evaluated in one masked forward pass. The correctness-critical constraint is that a noisy block may attend only to *strictly previous* clean blocks and never to its own clean copy.

Implemented training-time adaptation. The completed M3 run did not optimize the ECLD decomposition in Eq. (14). The source M1 checkpoint had been trained with the original Lagrangian form of Categorical Self-Distillation (CSD), so changing the distillation objective during fine-tuning would not have produced a matched adaptation. M3 consequently combined the diagonal VFM cross-entropy with the same Lagrangian/CSD objective, using disjoint subsets of each batch for the two terms. We report this distinction explicitly: M3 tests the block-conditional implementation path, but it is not an empirical test of the proposed ECLD objective.

Inference-time adaptation. The training-free variant skips Eq. (14) entirely and reuses a pretrained full-sequence CFM as the head $q^{\theta}_{s,t}$, feeding it the discrete prefix as conditioning and running the same blockwise sampler below. This construction tests how much semi-autoregressive behavior can be obtained without additional optimization and provides the training-free comparison required for H1.

Algorithm 1 One-step-per-block BCFM sampling.

Require: trained head q^{θ}, block size B, length L, prior p_0
Ensure: sequence $x = (x^{(1)}, \ldots, x^{(L/B)})$

```
c ← ∅                                   ▷ empty prefix / KV cache
for b = 1 to L/B do
    z_0^(b) ~ p_0^⊗B                    ▷ block prior on the simplex
    z_1^(b) ← X^θ_{0,1}(z_0^(b); c)     ▷ single jump 0→1
    x_i^(b) ← arg max_k [z_1^(b)]_{i,k},  i = 1, ..., B   ▷ discretize
    c ← c ∥ x^(b)                       ▷ append finalized block
end for
return x = (x^(1), ..., x^(L/B))
```

Blockwise sampling. Both variants share one loop. Starting from an empty cache, for each $b = 1, \ldots, L/B$ we draw $z_0^{(b)} \sim p_0^{\otimes B}$; apply the flow map for S steps following a schedule (e.g. $\{(0,1)\}$ or $\{(0,\frac{1}{2}),(\frac{1}{2},1)\}$) conditioned on $c^{(b)}$; discretize the endpoint to $x^{(b)}$; and append it to the prefix/cache. Algorithm 1 specifies the one-step-per-block case (a single jump $X_{0,1}$ per block, i.e. $n = 0$ in Eq. (6)). Since each block conditions only on the *clean* prefix, finalized blocks can form a reusable KV cache and the process can append an arbitrary number of blocks. Whether caching reduces end-to-end latency depends on both B and S, as well as on the cost of constructing prefix states. The principal risks are error accumulation and exposure bias, since at inference $c^{(b)}$ contains *generated*, rather than gold, blocks.

4 Experiments and Results

4.1 Experimental design

Models and hypotheses. All experiments use GPT-2-tokenized TinyStories and generate $L = 256$ tokens. The CFM-family network has hidden width 384, six transformer blocks, six attention heads, and vocabulary size 50,257. **M1-source** is the available full-sequence checkpoint at step 68,001; **M1-200k** is the continued full-sequence baseline at exactly 200,000 steps. **M2** applies M1-source blockwise without training, while **M3** fine-tunes the same source checkpoint as a $B = 16$ block-conditional model for 100,000 additional optimization steps. We test **H1**: learned block conditioning outperforms the training-free wrapper, and **H2**: BCFM improves the full-sequence CFM quality–latency frontier. The matched-scale MDLM and $B = 16$ BD3-LM checkpoints serve as discrete-diffusion references.

Training. M3 uses a micro-batch of 32 with four-step gradient accumulation (effective batch 128), AdamW with learning rate 10^{-4} and weight decay 0.01, BF16 mixed precision, and gradient clipping at 1.0. The effective update allocates 96 examples to VFM and 32 to Lagrangian/CSD, with unit loss weights. Thus the run evaluates block-conditional adaptation, but not the target ECLD objective in Eq. (14).

Quality protocol. Each plotted point uses 512 generated sequences at seed 0. MAUVE is computed from `gpt2-large` features against a fixed cache of 2,048 TinyStories reference sequences, with context truncated to 256 tokens. Generative perplexity (gen-PPL) is scored by `EleutherAI/gpt-j-6B`. CFM endpoints use argmax discretization, as this is the model's primary deterministic decoding mode; MDLM and BD3-LM use their canonical stochastic samplers. Reporting both metrics is essential: low external-model PPL alone can reward a narrow or collapsed generator.

Latency protocol. Sequence latency is measured on one NVIDIA H100 NVL at batch size one and length 256 in FP32, after five warm-up generations, using the median of 30 repeats; the horizontal error bars show the 10th–90th percentile range. Every model uses the same synchronization and sequence-timing harness copied from upstream commit `2e21c577`. NFE denotes flow steps for CFM/BCFM and sampler steps for the diffusion baselines; the corresponding total network forwards are listed separately. CFM endpoints use argmax whereas MDLM and BD3-LM retain their canonical stochastic samplers, so latency is matched end to end rather than per-forward.

4.2 Full-sequence CFM baseline

Continuing full-sequence training materially improves M1. At NFE 32, MAUVE rises from 0.5067 for the step-68k source checkpoint to 0.6577 at step 200k, while gen-PPL falls from 90.41 to 67.49. Latency changes only from 125.74 to 133.79 ms. The continued checkpoint is therefore the appropriate full-sequence reference for H2. Its best low-latency point is NFE 16 (67.99 ms, MAUVE 0.5229, gen-PPL 66.48); NFE 32 trades roughly twice the latency for +0.135 MAUVE.

4.3 Training-free and learned block conditioning

M2 fails on TinyStories. Even at NFE 32 (512 blockwise forwards), it reaches only MAUVE 0.0267 and gen-PPL 223.68 in 367.03 ms. The source full-sequence model was never trained to denoise a suffix conditioned on a discretized clean prefix, and the resulting train–inference mismatch is consistent with this collapse. This rules out the training-free wrapper as a competitive BCFM implementation in the tested configuration. Importantly, M2 and M3 now use the same cached/compiled sampler: at NFE 16 their latencies are 189.19 and 190.09 ms. The old full-backend M2 measurement was 1793.50 ms, so its apparent order-of-magnitude latency gap was an implementation artifact rather than an effect of fine-tuned weights.

M3 reverses the quality failure. Its best measured point is NFE 16: MAUVE 0.7550, gen-PPL 58.13, and 190.09 ms latency. Relative to M1-200k at NFE 32, M3 gains 0.0972 MAUVE and lowers gen-PPL by 9.36, at 1.42× the latency. It therefore extends the CFM-family frontier but does not strictly dominate M1. H1 is supported for generation quality, while H2 is supported only as a frontier extension rather than a simultaneous quality-and-latency win.

Refinement is non-monotonic. M3's MAUVE rises from 0.2821 at NFE 8 to 0.7550 at NFE 16, then falls to 0.6443 at NFE 32 while latency nearly doubles. Moreover, M3's lowest gen-PPL occurs at NFE 4 (46.11), where MAUVE is only 0.0334. This disagreement shows why gen-PPL cannot be interpreted without a distributional metric.

Maximum-speed backend. The original doubled-sequence M3 sampler recomputed all clean and noisy positions for every block step. At NFE 16 it measured 1830.44 ms and MAUVE 0.7979. The reported fast backend restricts queries to the current block, caches finalized-prefix keys and values, and compiles the repeated update; it reduces latency to 190.09 ms. It is not bitwise-identical because dense SDPA and the original FlexAttention path use different reduction orders. We therefore pair the fast sampler with its independently regenerated MAUVE 0.7550 rather than transferring the old quality score.

4.4 Comparison with discrete diffusion

The canonical diffusion baselines remain stronger in absolute quality and, for MDLM, on the matched quality–latency frontier. MDLM NFE 16 obtains MAUVE 0.9157 and gen-PPL 26.99 in 127.97 ms. It strictly dominates both M3 NFE 16 (190.09 ms, MAUVE 0.7550, gen-PPL 58.13) and M1-200k NFE 32 (133.79 ms, MAUVE 0.6577, gen-PPL 67.49). MDLM reaches MAUVE 0.9446 and gen-PPL 17.14 at NFE 64 in 482.93 ms. BD3-LM NFE 16 has the highest measured MAUVE, 0.9507, but takes 1508.32 ms; relative to MDLM NFE 64, it gains only 0.0061 MAUVE at 3.12× latency and with worse gen-PPL (21.51). Thus M3 extends only the CFM-family frontier, not the global frontier over the tested methods.

4.5 Limitations

The quality grid uses one generation seed and 512 samples per point, so MAUVE uncertainty is not estimated. The latency harness and hardware are matched, but the comparison remains end to end: CFM uses deterministic argmax endpoints whereas MDLM and BD3-LM use their canonical stochastic samplers and distinct model code. M2 and M3 share a separately validated maximum-speed sampler. This sampler preserves the block-causal mask algebra but not bitwise trajectories relative to the full FlexAttention backend, so both curves use independently regenerated quality scores. The experiment studies only $B = 16$; there is no trained TinyStories BCFM checkpoint for $B = 64$. A fuller frontier requires repeated quality evaluation across seeds, additional trained block sizes, and a bitwise-controlled fast sampler.

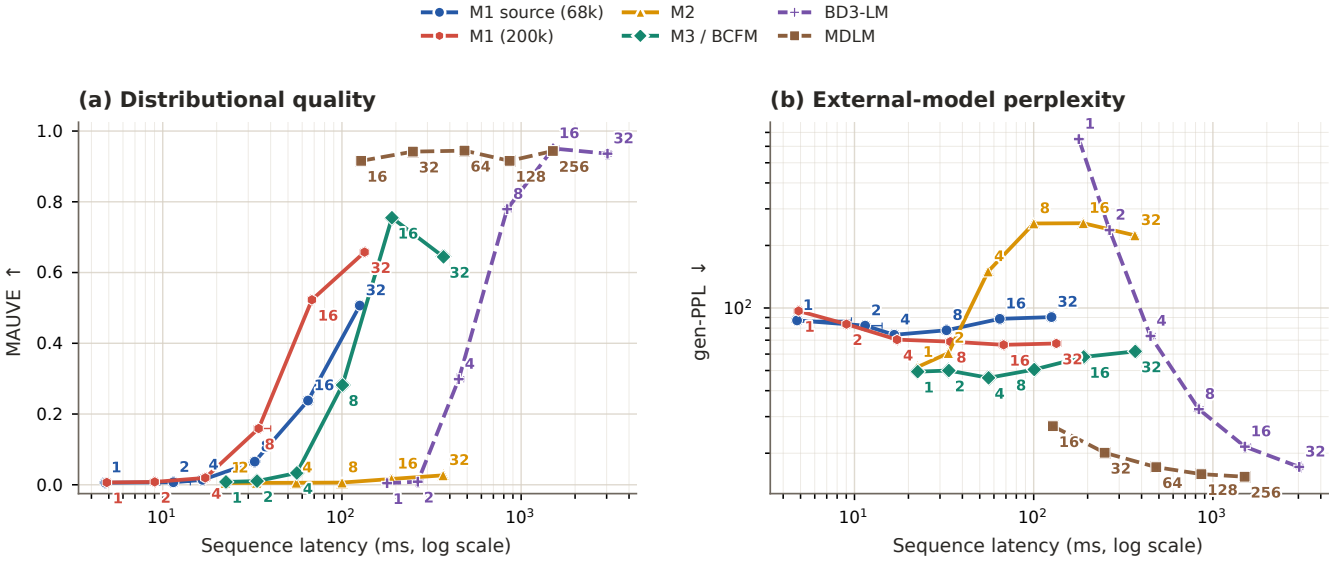

Figure 1. TinyStories quality versus batch-one sequence latency on one H100 NVL. Numbers next to points are NFE. Solid curves use argmax CFM-family endpoints; dashed curves use the canonical stochastic MDLM/BD3-LM samplers. All points use the same 512-sample quality protocol and the same batch-one FP32 latency protocol.

Table 1
Selected TinyStories operating points from Fig. 1. Higher MAUVE and lower gen-PPL/latency are better; all latency is measured on one H100 NVL.

Family	Checkpoint / sampler	NFE	Forwards	MAUVE	gen-PPL	Latency (ms)
CFM family, H100 NVL, argmax						
M1	source, step 68k	32	32	0.5067	90.41	125.74
M1	step 200k	16	16	0.5229	66.48	67.99
M1	step 200k	32	32	0.6577	67.49	133.79
M2	fast, source, $B = 16$	16	256	0.0164	255.90	189.19
M2	fast, source, $B = 16$	32	512	0.0267	223.68	367.03
M3	fast, $B = 16$	8	128	0.2821	50.57	100.71
M3	fast, $B = 16$	16	256	**0.7550**	**58.13**	190.09
M3	fast, $B = 16$	32	512	0.6443	61.84	368.82
Discrete-diffusion references, H100 NVL, canonical samplers						
MDLM	step 100k	16	17	0.9157	26.99	127.97
MDLM	step 100k	64	65	0.9446	17.14	482.93
BD3-LM	step 100k, $B = 16$	8	144	0.7792	32.57	835.23
BD3-LM	step 100k, $B = 16$	16	272	**0.9507**	21.51	1508.32

5 Conclusion

BCFM combines an exact autoregressive factorization over blocks with few-step categorical flow maps inside each block. On TinyStories, merely wrapping a full-sequence CFM blockwise is ineffective: M2 remains near zero MAUVE even after hundreds of network forwards. Training the block condition changes the result qualitatively. M3 reaches MAUVE 0.7550 at NFE 16, improving over the 200k-step full-sequence CFM while remaining within a few hundred milliseconds per sequence on an H100.

The gain does not match the discrete-diffusion baselines: MDLM and BD3-LM exceed MAUVE 0.94 under the shared quality protocol, and MDLM NFE 16 strictly dominates the best-MAUVE M3 point under matched H100 timing. The supported conclusion is therefore narrow. H1 is supported—learned block conditioning is substantially better than the training-free wrapper—and M3 extends the CFM-only frontier, but it does not establish an overall quality–latency advantage. The most direct next experiment is a multi-seed comparison with additional trained block sizes and a bitwise-controlled fast sampler.

References

[1] Marianne Arriola et al. "Block diffusion: Interpolating between autoregressive and diffusion language models". In: *International Conference on Learning Representations*. 2025.

[2] Tiwei Bie et al. "Llada2. 0: Scaling up diffusion language models to 100b". In: *arXiv preprint arXiv:2512.15745* (2025).

[3] Floor Eijkelboom et al. "Variational flow matching for graph generation". In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 11735–11764.

[4] Google DeepMind. *DiffusionGemma*. `https://deepmind.google/models/gemma/diffusiongemma/`. Open-weights text-diffusion language model. Accessed 2026-06-17. June 2026.

[5] Aaron Grattafiori et al. "The llama 3 herd of models". In: *arXiv preprint arXiv:2407.21783* (2024).

[6] Chanhyuk Lee et al. "One-step Language Modeling via Continuous Denoising". In: *arXiv e-prints* (2026).

[7] Aaron Lou, Chenlin Meng, and Stefano Ermon. "Discrete diffusion modeling by estimating the ratios of the data distribution". In: *Proceedings of the 41st International Conference on Machine Learning*. 2024, pp. 32819–32848.

[8] Peter Potaptchik et al. "Discrete flow maps". In: *arXiv preprint arXiv:2604.09784* (2026).

[9] Alec Radford et al. *Language models are unsupervised multitask learners*. Tech. rep. OpenAI, 2019.

[10] Daan Roos et al. "Categorical flow maps". In: *arXiv preprint arXiv:2602.12233* (2026).

[11] Subham S Sahoo et al. "Simple and effective masked diffusion language models". In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 130136–130184.

[12] Subham Sekhar Sahoo et al. "The Diffusion Duality". In: *International Conference on Machine Learning*. PMLR. 2025, pp. 52584–52619.